

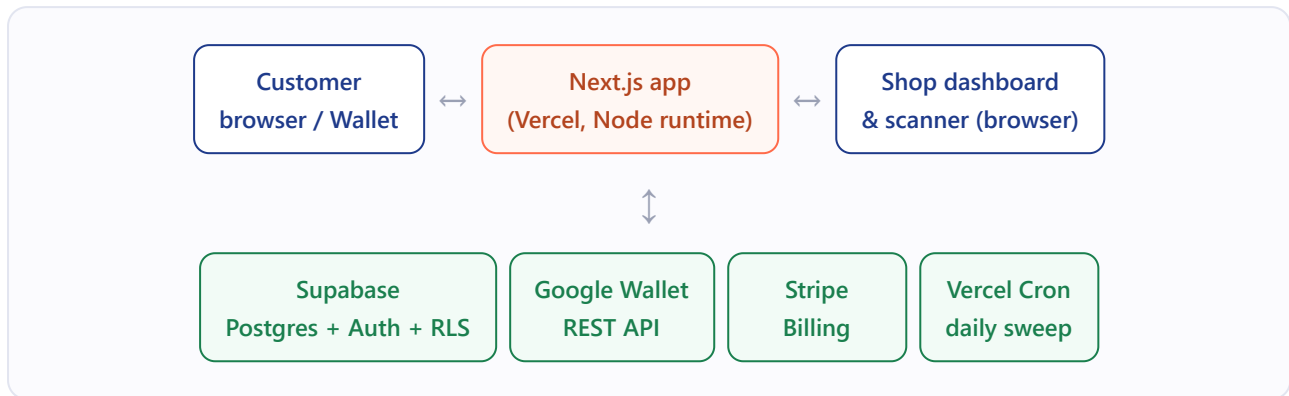
HIGH-LEVEL DESIGN

Jeeran Network

System architecture, major components, and how the core flows move data between them.

Version 1.0 · August 2026

1. Architecture Overview



Single Next.js application (App Router) deployed on Vercel, using Node.js Functions throughout (no Edge runtime dependency). All persistent state lives in one Supabase Postgres database, accessed two ways:

- **Session-scoped client** — used in server components / API routes acting on behalf of a logged-in shop owner; Postgres Row-Level Security enforces "you can only touch your own rows."
- **Service-role client (`supabaseAdmin`)** — used for cross-business reads (admin views, leaderboards, metrics) and anything touching customer wallet data, which is deny-by-default under RLS and never exposed to a browser session directly.

2. Major Components

Component	Responsibility
Matchmaking engine	Decides which offers a given viewer sees — <code>get_top_ads()</code> for the scan page, <code>nearby_active_offers()</code> for the wallet geo-push, <code>browse_businesses_by_category()</code> for the public directory. Three separate ranking rules for three separate contexts, sharing one tier-configuration source.
Wallet pass service	Owns the Google Wallet class/object lifecycle — creation, localization, geofencing points, card refresh, and typed lock-screen notifications.
Geo-push pipeline	Decides <i>when</i> a pass needs refreshing: event-driven on campaign changes, plus a daily cron safety net for anything with no explicit trigger (lapsed dates, expired subscriptions).
Redemption service	Validates a scanned pass, checks offer eligibility, records the redemption, and triggers every downstream credit consequence of that one event.
Rewards & attribution engine	Owns three distinct credit-earning events (signup bonus, rescan credit, milestone bonus) plus the permanent attribution model, each with its own anti-farming guard.
Promotion/tier engine	Computes tier from redemption score on demand; supplies the shared, admin-configurable thresholds every ranking function reads.
Billing	Stripe Checkout for subscriptions/top-ups, webhook-driven credit issuance, trial lifecycle cron.
Admin console	Cross-shop operational visibility and platform-wide configuration (currently unauthenticated — a tracked gap, not an oversight).

3. Core Flow — Customer Discovers & Saves

CUSTOMER DEVICE

Scans shop's printed QR → lands on `/scan/[business_id]` → sees live nearby offers → taps "Add to Google Wallet"

JEERAN SERVER

`/api/wallet/membership/create` — resolves home location (browser geolocation or shop fallback), creates (or reuses) the customer's `wallet_members` row, pays the one-time signup bonus if new, computes initial nearby offers, creates the Google Wallet object

RESPONSE TO CUSTOMER

New customer: a warm "You're in!" confirmation, then the real Google Wallet save prompt. Returning customer: straight to the save prompt (no repeated moment)

4. Core Flow — Offer Changes Reach the Pass

TRIGGER

A shop creates, edits, activates, or deactivates a campaign — **or** nothing happens at all (a date silently lapses, a subscription silently expires)

EVENT-DRIVEN PATH

The campaign API route fires `notifyMembersNearBusiness()` for every explicit change → finds members within a broad radius → each member's own `nearby_active_offers()` is recomputed and diffed against what they were last shown

SAFETY-NET PATH

A daily Vercel Cron sweep (`/api/cron/wallet-refresh`) recomputes every member's offers regardless of trigger — catches everything the event-driven path structurally can't see

PUSH

Only members whose offer set actually changed get a real Wallet API call — card content always, a typed lock-screen notification only if a genuinely new offer appeared

5. Core Flow — Redemption & Its Consequences

STAFF

Scans the customer's pass barcode (or types it) on `/dashboard/redeem` → `/api/redeem`

VALIDATION

Decode & verify the signed token → confirm the pass is still saved (not deleted) → confirm the shop has a live, unredeemed offer for this customer

ON SUCCESS, THREE INDEPENDENT EFFECTS FIRE

1. Redemption logged (blocks re-claiming)
2. Ad-credit transfer to the customer's *permanent* recruiting shop
3. Tier-milestone check — pays a one-time bonus and flags a dashboard announcement if this redemption pushed the shop into a new tier

All three are best-effort relative to each other — a failure in the credit transfer or milestone check is logged for reconciliation, never rolled back onto the customer's already-successful redemption.

6. Technology Stack

Application

Next.js (App Router), TypeScript, Node.js runtime throughout, Vercel hosting + Cron

Data

Supabase (Postgres + PostGIS for geography, Auth, Row-Level Security)

Integrations

Google Wallet REST API, Stripe (Checkout + webhooks), SMTP for trial-lifecycle email

Client-side

Native `BarcodeDetector` where available, `jsQR` fallback for Safari/iOS, browser Geolocation API

7. Deployment & Operational Model

- **Single production environment**, deployed via Vercel from the repository's `master` branch.
- **Two scheduled jobs**: a keepalive ping (prevents the free-tier database from pausing) and the wallet-refresh sweep — both once-daily, a constraint of the current hosting tier rather than a design choice.
- **Database migrations** are additive, hand-authored SQL files applied directly against the live project; every ranking-affecting function change in this project's history was verified against real data before being trusted, not just read back.
- **No separate staging environment today** — changes are tested against real (and clearly-marked test) data in the same production database, then deployed straight to production.

8. Non-Functional Considerations

Concern	Approach
Scalability	Matchmaking queries are radius-bounded (PostGIS <code>ST_DWithin</code>) and limit-capped; the geo-push sweep only re-pushes members whose offer set actually changed, not the full population every time.
Consistency	Credit-affecting operations that must not double-fire (milestone bonus, ad-credit transfer) use row-level locking (<code>FOR UPDATE</code>) inside dedicated Postgres functions, not read-then-write from the application layer.
Internationalization	Arabic-first, with English and Urdu — both the shop dashboard (full UI) and the customer wallet pass (fixed labels, OS-resolved) are localized; offer content itself is not translated.
Privacy	No server-side live location tracking of customers; geofencing is delivered entirely on-device by the Wallet platform.
Abuse resistance	Daily per-shop caps and per-pair cooldowns on every credit-earning mechanic, with IP logged for manual review — an accepted, not eliminated, residual risk.